Platzhalter

Lo
go

Hausarbeit

Beispieltitel
für einen Bericht

Autor:	Vorname Nachname
Matrikel-Nr.:	xxxxxxx
Studiengang:	Maschinenbau
Erstprüfer:	Prof. Dr. Elmar Wings
Abgabedatum:	20. Februar 2026

Hochschule Emden/Leer · Fachbereich Technik · Abteilung Maschinenbau
Constantiaplatz 4 · 26723 Emden · `http://www.hs-emden-leer.de`

Inhaltsverzeichnis

Inhaltsverzeichnis i

Abbildungsverzeichnis v

Liste des Listings vii

Acronyms ix

I. How to: Text schreiben 1

1. Text Skills, auf die es ankommt 3
1.1. Allgemein . . . 3
1.2. Einleitungen . . . 3
1.3. Beschreiben Sensoren und Aktoren . . . 4

II. How to: Ordnerstruktur 5

2. Ordnerstruktur erklärt 7
2.1. Git-Ordner . . . 7
2.2. Projektordner . . . 7
2.2.1. Code . . . 8
2.2.2. System . . . 8
2.2.3. Contents . . . 9
2.2.4. Images . . . 9
2.2.5. General . . . 9
2.3. `Readme.md` . . . 10
2.4. Assumption of Liability . . . 10
2.5. Assembly . . . 10
2.6. Manual . . . 10
2.7. Presentations . . . 11
2.8. Poster . . . 11
2.9. ProjectManagement . . . 11
2.9.1. Evaluation Sheet und Resulting Objects . . . 11
2.10. MLbib . . . 12

III. Befehle erklärt 13

3. Benutzerdefinierte Befehle: 15

4. Befehle von LaTeX: 17

IV. Acronyms 19

5. Einbinden von Acronymen 21

V. Literatur 23

6. Literatur einbinden 25
6.1. Literatur-Sammlung 25
6.2. Literature Review und Keyword Auflistung 26
6.3. `MyLiterature.bib` 26
6.4. Literatur in den Text einbinden 27

VI. Code 29

7. Code in den Bericht Integrieren 31
7.1. Beispiel zur Integration eines Codes 31
7.2. Beispiel zur Integration eines Codeabschnitts 31

VII.Bilder 33

8. Einbinden von Bildern 35

VIIITikz 37

9. Implementieren von Tikz Figures 39
9.1. Einbinden von Tikz in den Text 39

IX. Tabellen 41

10.Tabellen in Text einbinden 43

X. BOM und SBOM 45

11. Bill of Materials 47

12. Software Bill of Materials 49

XI. Templates 51

13. Sensor XY/ Actor XY 53

13.1. General 53
13.2. Specific Sensor/Actor 53
13.3. Specification 53
13.4. Library 53
13.4.1. Description 53
13.4.2. Installation 53
13.4.3. Functions 53
13.5. Example 53
13.5.1. Manual 54
13.5.2. Inside the Example 54
13.5.3. Code 54
13.5.4. Files 54
13.6. Calibration 54
13.7. Simple Code 54
13.8. Simple Application 54
13.9. Tests 54
13.9.1. Simple Function Test 54
13.9.2. Test all Functions 54
13.10 Further Readings 54

14. Sensor XY/ Aktor XY 55

14.1. Allgemeines 55
14.2. Spezifischer Sensor/Aktor 55
14.3. Spezifikation 56
14.4. Bibliothek 56
14.4.1. Beschreibung 56
14.4.2. Installation 56
14.4.3. Funktionen 56
14.5. Beispiel 56
14.5.1. Anleitung 56
14.5.2. Innerhalb des Beispiels 56
14.5.3. Code 56
14.5.4. Dateien 56
14.6. Kalibrierung 56
14.7. Einfacher Code 57

14.8. Einfache Anwendung 57
14.9. Tests 57
14.9.1. Einfacher Funktionstest 57
14.9.2. Test aller Funktionen 57
14.10 Weiterführende Literatur 57

Literaturverzeichnis **59**

Index **61**

Abbildungsverzeichnis

Liste des Listings

Abkürzungen

Abb Abbildung

Teil I.

How to: Text schreiben

1. Text Skills, auf die es ankommt

1.1. Allgemein

- Ganz wichtig über das gesamte Projekt: Mitschreiben von
  - ⇒ Welche Probleme/ Aufgaben wurden gelöst?
  - ⇒ Welche Herausforderungen gab es am Anfang?
- Behauptungen wie: Der Sensor wird häufig angewendet braucht mindestens 3 Literaturnachweise zur Bestätigung
- Neutrale wissenschaftliche Schreibweise: keine Werbung für einen Sensor machen - neutral bewerten und vergleichen
- Konstante Bezeichnungen einführen: Die "ON/OFF LED", Die "User Feedback LED", Die "grüne LED" ⇒ nicht wechseln zwischen den Begrifflichkeiten, sondern für einen entscheiden (Eindeutigkeit!)
- Immer erst die Funktion eines Namens zuerst, dann die Kennzeichnung Bsp.: Die Bibliothek HX711, die Funktion `GetSensorData`, die Kamera OV7675, usw.
- Klare Trennung zwischen eigenem Projekt und allgemeine Infos (Im Text wie auch in der Ordnerstruktur)

1.2. Einleitungen

- Projektlegitimation, welchen Hintergrund/ Beitrag zur Gesellschaft ... hat das Projekt
- Die Einleitung ist ähnlich einer Marktrecherche

  ⇒ Was ist der State of the Art meines Projektes/ meines Sensors/ Aktors
- Wurde das Projekt schon irgendwo angewendet?
- Welche Herausforderungen sind laut anderen ähnlichen Projekten oder Ressourcen zur Hardware etc. zu erwarten?

- Jedes Statement muss mit einer Literatur Angabe versehen werden!!!
- Struktur:
  - Problembeschreibung
  - Herausforderungen
  - Struktur des Berichts

1.3. Beschreiben Sensoren und Aktoren

- Es gibt ein Template für Sensoren und Aktoren, in der Ordnerstruktur
- Bei einer BA ist das Template nicht optimal - was eigenes machen für Sensoren etc.
- Es gibt Hardware, für die das Template nicht geeignet ist. Sensoren und Aktoren beschreibt die Hardware, die direkt von einem Mikrocontroller ausgelesen werden. Sprich ein Motor ist kein Aktor, sondern der zugehörige Motortreiber. Hardware, wie Motoren, Gehäuse, Schrauben müssen ohne Template beschrieben werden.
- Falls ein Sensor oder Aktor nicht kalibriert sondern justiert werden muss, kann das Kapitel Kalibration der Templates zu Justage umbenannt werden.

Teil II.

How to: Ordnerstruktur

2. Ordnerstruktur erklärt

2.1. Git-Ordner

Der Ordner, der von GitHub gecloned wird, beinhaltet mindestens:

- Datei `Author.xlsx`
- Datei `Readme.md`
- Ordner:
  - Projektordner (Name abhängig vom Projekt)
  - Assumption of Liability
  - Assembly
  - Manual
  - Presentations
  - Poster
  - Code
  - ProjectManagement

2.2. Projektordner

Der Projektorder ist die Spielwiese des Projekts. Die Struktur im Projekt umfasst:

- Code
- System
- Contents
- Images
- General

Die TeX-Dateien, die in die Main integriert werden, liegen unter Contents und können jeweils Elemente aus Code und Images enthalten. In General werden allgemeine Dokumenteneigenschaften festgelegt.

2.2.1. Code

Hier kann Code hinterlegt werden, um diesen in Dokumenten aufzurufen. Es gilt eine Element orientierte Struktur. Beispiele:

- `Code/Projektname/ProjektnameSystem.ino`
- `Code/LED/TestLED.ino`
- `Code/LED/ApplikationLED.ino`
- `Code/Kamera/TestKamera.ino`
- `Code/Kamera/ApplikationKamera.ino`
- usw.

2.2.2. System

Der wichtigste Ordner ist der Ordner `System`. Hier liegt die Datei `main.tex`, die den gesamten Bericht enthält. Die TeX-Datei heißt zu Beginn `rename`. Der Name ist Programm ⇒ alle rename-Dateien sollen entsprechend des Projekts umbenannt werden. Die Datei `main.tex` ist folgendermaßen strukturiert:

- Der mit % markierte Bereich beschreibt die Datei und darf entsprechend des Projekts angepasst werden. Dieser Header sollte in sämtliche Content-Dateien kopiert werden.
- `\def\isGerman{1}` und `\def\isEnglish{1}` kann je nach Sprachauswahl auskommentiert werden. Dies hat Auswirkungen auf `InputLanguage` im Bereich documents
- Alles oberhalb `begin{document}` beschriebt die Dokumentenklasse und integriert verschiedene Dokumenteneigenschaften. In diesem Bereich sollte nichts verändert werden außer der Sprachauswahl
- Der Bereich zwischen `begin` und `end document` integriert die Dateien des Projektes. Hier können alle Dateien und Veränderungen vorgenommen werden.

  Teile, die nicht verwendet werden, können mit Hilfe von % auskommentiert werden.

  Außerdem werden hier die Standard Abschnitte wie Inhaltsverzeichnis, Literaturverzeichnis, etc. eingebunden. Diese sollten bestehen bleiben und nicht verändert werden.
- `input{../../Contents/...}` integriert eine Datei aus Contents

- `InputLanguage{../../Contents/}...` integriert eine Datei aus `Contents/en` oder `Contents/de` je nach vorheriger Sprachauswahl.

2.2.3. Contents

- In Contents können neue Element orientierte Ordner erstellt werden, in denen sich die Dateien zum Bericht befinden. Bsp.:
  - `Contents/Kamera/InbetriennahmeOV7675`
  - `Contents/Kamera/SensorOV7675`
  - `Contents/Projektname/Einleitung`
  - `Contents/Projektname/Applikation`
  - `Contents/en/ArduinoIDE` (englische Version)
  - `Contents/de/ArduinoIDE` (deutsche Version) (en und de betrifft vorhandene Dateien ⇒ hier keine neuen Dateien erstellen)
  - usw.

General

In General sind verschiedene Sensoren beschrieben. Eine Suche nach brauchbaren Dateien für das Projekt lohnt sich. Außerdem sind hier die Templates für Sensoren und Aktoren abgelegt. (Hier keine neuen Dateien erstellen)

2.2.4. Images

In der Ordnerstruktur `Images` werden alle im Bericht verwendeten Bilder Element orientiert abgelegt. Beispiel:

- `Images/Projektname/ProjektSetup.png`
- `Images/LED/LEDON.png`
- `Images/Kamera/KameraSetup.png`
- usw.

2.2.5. General

Wichtig für den Bericht sind die im Ordner `General` abgelegten Dateien `acronyms.tex` und `tikzdefs.tex`. Diese dienen zum Einbinden von Akronymen und Tikz und können entsprechend ergänzt werden. Die restlichen Dateien dienen dem Dokumentaufbau und bleiben unangetastet.

2.3. `Readme.md`

Hier wird das Projekt vorgestellt. Dieser Text erscheint auch im GitHub als Projektbeschreibung.
Markdown-Code-Beispiel für `Readme.md`:

- `#` Titel
- `##` Untertitel
- `###` Untertitel
- Einbinden eines Logos: `![Projekt Logo](report/Images/General/ProjektLogo.pn`
- Link zu Überschriften fürs Inhaltsverzeichnis: `- [Overview](#overview)`
- Stichpunkte: `- Text`
- `**text fett**`
- Grauer Balken: `--`
- Grau hinterlegt mit Kopierfunktion (Code oder Terminal-Befehle): `“‘bash`

2.4. Assumption of Liability

- Assumption of Liability verpflichtend für die Hardwareausgabe. Tipp: Durchstreichen von nicht ausgegebenen Teilen

2.5. Assembly

- Zum Assembly gehören Montageanleitung und Demontageanleitung
- Montageanleitung entspricht Entwicklerhandbuch
- Bei jedem Montage-Schritt muss die Demontage im Hinterkopf bleiben: KEINE HARDWARE KLEBEN!

2.6. Manual

- Das Handbuch ist ausgelegt auf das Verständnis und die Interessen des Kunden ⇒ keine Beschreibung der System-Hardware, sondern eine Darstellung des Produktumfangs und der Inbetriebnahme des ausgelieferten Systems

2.7. Presentations

- Vorhandenes Template für Präsentationen mit Beispielen für Tikz etc. zur Orientierung

2.8. Poster

Projektposter sollten mindestens enthalten:

- Inhalt: Orientierung an der Einleitung zum Projekt
- Nicht mehr als 20% Bildmaterial
- Keine weißen Leerstellen/Lücken ⇒ Ausfüllen der vorhandenen Oberfläche
- Bild: Produktumfang
- Einstieg zur Verwendung des Produkts
- Herausforderungen, die bewältigt wurden
- Darstellung der erzielten Anforderungen
- Generell gilt: Orientierung am vorhandenen Poster-Template: Kein Fokus auf Stilveränderung!

2.9. ProjectManagement

Der Ordner `ProjektManagement` dient zur Organisation des Teams und zur Orientierung der Projektanforderungen.

2.9.1. Evaluation Sheet und Resulting Objects

- Das Evaluation Sheet ist ein grundsätzlicher Leitfaden durch das Projekt
- Resulting Objects beschreibt im Detail, was möchte ich wo machen, um es besser zu machen? Qualitätscheck am Ende
- Tipp: Evaluation Sheet einkopieren in Resulting Objects und abhaken, was gemacht wurde

2.10. MLbib

In MLbib sind alle Dokumente zur Literatur Verwaltung untergebracht. Dabei werden sämtliche Dateien im pdf-Format in Element orientierten Ordnern gespeichert. Die hier vorhandene Literatur kann für das Projekt verwendet werden und es sollten sämtliche weitere Quellen hier ergänzt werden.
Die vorhandenen Dateien:

- `Jetson.bib`
- `Literature.bib`

Sollten unverändert bleiben und dienen zur Orientierung. Die hier angegebene Literatur kann ebenso in die Dokumentation eingebunden werden. Die Datei `MyLiterature.bib` dient der Referenz aller Literaturquellen des Projektes. Es gilt: Keine weitere `.bib` zu erstellen, sondern die vorhandene `MyLiterature.bib` ergänzen.

Teil III.

Befehle erklärt

3. Benutzerdefinierte Befehle:

Die hier für die Projekte entwickelten Befehle sind notwendigerweise in allen Texten und Beschreibungen zu integrieren.
Sämtliche Syntax-Beschreibungen im Text werden durch `\PYTHON{}` markiert. Es sei denn, es handelt sich um ein Terminal-Befehl. In diesem Fall wird \SHELL{} eingesetzt.
Dateipfade werden mit `\FILE{}` gekennzeichnet. Dabei können Beschreibungen von einer Menüauswahl durch \menu{button > button > button} verdeutlicht werden.
Links in BOM oder als Referenz zu GitHub-Projekten (Bibliotheksbeschreibungen) oder Ähnlichem können ergänzt werden über den Befehl `\URL{}` oder `\HREF{}{}`.

4. Befehle von LaTeX:

Auf sämtliche Bilder und Listings sollte im Text eingegangen werden, aber auch auf andere Kapitel kann sich im Text bezogen werden. Die dabei verwendeten Referenzen werden mithilfe von
`\ref{fig:bild}`, `\ref{lst:code}` bzw. `\ref{Kapitel}`
realisiert.
Die dazugehörigen Labels werden erstellt durch:
`\label{fig:bild}`, `\label{lst:code}` bzw. `\label{Kapitel}`.

Teil IV.

Acronyms

5. Einbinden von Acronymen

Im Text können Abkürzungen erstellt werden. Beispiel: \ac {abb}. Dadurch erscheint bei der ersten Verwendung: Abkürzung (Abb). Bei allen weiteren Verwendungen erscheint: Abb.
Erster Aufruf: Abbildung (Abb) zweiter Aufruf: Abb
In die Datei `report/General/acronyms` wird alphabetisch sortiert folgendes eingefügt:

```
\DeclareAcronym{abb}{
short = Abb \index {Abb},
long = Abbildung \index{Abbildung! \Siehe {Abb}},
}
```

Durch `\index {Abkürzung! \Siehe {ABB}}` wird die Abkürzung automatisch im Index-Verzeichnis aufgelistet.

Teil V.

Literatur

6. Literatur einbinden

Gute Literatur kennzeichnet sich unter anderem aus durch:

- Wissenschaftlich zitierfähig (Fachbücher, Konferenzbeiträge, offizielle Normen)
- Autoren oder klare institutionelle Zuordnung
- Veröffentlichungsdatum angegeben und thematisch aktuell
- Primärquelle bevorzugen (Originalarbeit statt Sekundärzitat)
- Dauerhaft zugänglich (pdf ablegen!)
- Keine nicht überprüfbaren Online-Quellen oder Forenbeiträge
- Kein Wikipedia als zitierfähige Quelle (nur zur Orientierung nutzen)
- Reichelt oder andere Anbieter sind keine Quelle für Datenblätter (immer Original-Hersteller verwenden)

6.1. Literatur-Sammlung

Zu Projektbeginn sollte sich ausgiebig mit Literatur auseinandergesetzt werden, um den State-Of-The-Art kennenzulernen und eine Übersicht zu erhalten, womit sich das Projekt beschäftigt und auf welchem Stand der Technik das Projekt aufbaut.
Die dabei gefundenen `pdf`-Dateien werden Element orientiert im Ordner `MLbib` abgelegt. Bsp.:

- `MLbib/KameraOV7675/DatenblattOV7675`
- `MLbib/Projektname/ResearchPaperXY`
- usw.

6.2. Literature Review und Keyword Auflistung

Diese abgelegten Dateien werden in eine Präsentation `Presentations-/LiteratureReview` eingebunden, um eine Übersicht der erarbeiteten Themen zu erhalten und zu einem späteren Zeitpunkt auf Inhalt einfacher zugreifen zu können. Dies vereinfacht die Teamarbeit als auch das Schreiben von Dokumentation im späteren Verlauf des Semesters.
Für jede Literaturquelle wird dabei auf einer Seite beschrieben, welche Themen die Literatur beinhaltet und welche Limitationen gegeben sind. Dabei wird immer eine Referenz zur Literatur implementiert. So entsteht in der Präsentation ein Literaturverzeichnis, was die eigene Qualitätskontrolle der Literatur vereinfacht. Außerdem werden auf unter einem Punkt Keywords die enthaltenen Themen als Stichpunkte aufgelistet. Alle Keywords sollten in eine Liste eingetragen werden und an das Ende der Präsentation angehängt werden.

6.3. `MyLiterature.bib`

Um in Texten Literatur zu zitieren, werden die gesammelten Dokumente in die `MLbib/MyLiterature.bib` alphabetisch sortiert eingebunden. Ein Beispiel [BS22]:

```
@book{Banzi:2022,
  title     = {Getting started with Arduino},
  author    = {Banzi, Massimo and Shiloh, Michael},
  year      = {2022},
  publisher = {Maker Media, Inc.},
  isbn      = {9780596555108}
}
```

Hier kommt es auf folgende Formdetails an:

- Die Referenz wird benannt nach Autor und Jahr wie hier im Beispiel `Banzi:2022`.
- Sollte ein Referenzbezeichnung bereits vorhanden sein, wie zum Beispiel `Arduino:2025`, ist es möglich die Bezeichnungen zur Buchstaben zu ergänzen: `Arduino:2025a`, `Arduino:2025b`, usw.
- Die Autoren werden aufgezählt und getrennt durch ein `and` Dabei werden zunächst die Nachnamen, dann die Vornamen erwähnt.
- Sollte es, zum Beispiel für Datenblätter, keinen Autor geben, sondern nur eine Institution, so wird `author` gegen `editor` getauscht und hier die Institution genannt. Ein Tipp ist jedoch, dass sich die Autoren und Daten der Veröffentlichung von pdf-Dateien manchmal im Browser herausfinden lassen über die drei Punkte in der oberen

rechten Ecke der geöffneten pdf : Dokumenteneigenschaften (Bespiel Chrome).

- In den Dateien `MLbib/Jetson.bib` und `MLbib/Literature.bib` befinden sich außerdem viele weitere Beispiele, die auch für das eigene Projekt verwendet werden dürfen. Diese müssen nicht erneut in `MyLiterature` eingebunden werden, da alle drei bib-Dateien in das Main-Dokument des Berichts eingebunden sind.

6.4. Literatur in den Text einbinden

Um im Text eine Referenz auf die in der `MyLiterature.bib` abgelegten Literatur einzubinden, werden folgende Befehle verwendet:

- `\cite Arduino:2025a` ergibt: [Ard25]
- Dies kann erweitert werden durch:
  - `\cite[23]{Arduino:2025}` ergibt: [Ard25, S. 23]
  - `\cite[23-25]{Arduino:2025}` ergibt: [Ard25, S. 23–25]
- Mehrere Zitate nacheinander sollten durch ein Komma getrennt angegeben werden:
  `\cite[23-25]{Arduino:2025},`
  `\cite[12-13]{Arduino:2023}`
  ergibt: [Ard25, S. 23–25],[Ard23, S. 12–13]

Teil VI.

Code

7. Code in den Bericht Integrieren

Um Code zu beschreiben, können entweder ganze Codes oder Code-Abschnitte in LaTeXintegriert werden. Die Dateien sollten dafür säuberlich in die Dateistruktur – z.B. unter `Projektname/Code/LED/LEDTest` – eingepflegt sein. Die beiden folgenden Beispiele beziehen sich auf die Inbuild LED des Arduino Nano 33 BLE Sense.
Beim Implementieren von Code ist darauf zu achten, dass Arduino-Dateien sich immer in einem gleichnamigen Ordner befinden müssen, sie würden sonst nicht kompilieren. Python Code hingegen ist auf eine bestimmte Einrückung der Code Zeilen angewiesen. Auch im Text sollte dies formal richtig dargestellt werden. Python Code wird genau, wie in den Beispielen implementiert, nur das statt `language=C++`, `language=Python` verwendet wird. Auf den Code sollte im Text eingegangen werden. Hierfür kann eine Referenz auf das Label zum Code gesetzt werden. Beispiel:
Die Inbuild LED leuchtet durch den Code `\ref{Nano:RGBLEDTest}` abwechselnd rot und grün.

7.1. Beispiel zur Integration eines Codes

```
\begin{center}
   \captionof{code}{Simple sketch to test the RGB LED}
   \label{Nano:RGBLEDTest}
   \lstinputlisting[language=C++]
   {../../Code/Nano33BLESense/Test/TestLEDRGB.ino}
\end{center}
```

7.2. Beispiel zur Integration eines Codeabschnitts

```
\begin{center}
   \captionof{code}{Snippet of RGB LED test}
   \label{Nano:RGBLEDTestSnippet}
   \lstinputlisting[language=C++, firstline=25, lastline=30]
```

```
   {../../Code/Nano33BLESense/Test/TestLEDRGB.ino}
\end{center}
```

Teil VII.

Bilder

8. Einbinden von Bildern

Bilder sollten Element orientiert in die Dateistruktur implementiert sein und können durch den folgenden Befehl in ein Dokument integriert werden:

```
\begin{figure}
   \begin{center}
      \includegraphics[width=\textwidth]
      {Battery/BatteryTest.png}
      \caption [Testoutput des Spannungssensors]{Der hier
      abgebildete Spannungsverlauf veranschaulicht das
      Verhältnis der Eingangspannung zur Ausgangsspannung
      des Sensors}
      \label{fig:BildSpannungTest}
      \end{center}
\end{figure}
```

Mithilfe von `\ref{fig:BildSpannungTest}` kann im Text eine Referenz auf das Bild erstellt werden.
Folgende Einstellungen können zum Bild getroffen werden:

- Die Größe des Bildes kann über `width=0.5\textwidth` verändert, hier zum Beispiel durch 0.5 auf die Hälfte der Textbreite reduziert werden.
- Die Größe des Bildes kann auch über die Höhe definiert werden: `height=0.5\textheight`.
- Die Bildunterschrift setzt sich zusammen aus zwei Elementen. Die eckigen Klammern definieren dabei die Bezeichnung des Bildes im Abbildungsverzeichnis und die geschweiften Klammern die Beschreibung des Bildes im Bericht. `\caption[]{}`
- Im allgemeinen sollten Formatierungen wie `\begin{figure}[H]` vermieden werden.

Teil VIII.

Tikz

9. Implementieren von Tikz Figures

Sämtliche Beispiele für die Erstellung von Tikz können der Datei `General/tikzdefs.tex` entnommen werden.

9.1. Einbinden von Tikz in den Text

In diesem Beispiel werden drei Tikzzeichnungen `\ArduinoNanoShieldTikz`, `\CameraOV` und `\ArduinoNanoBLESenseRev` übereinander gelegt. Diese werden über definierte Koordinaten (`\coordinate(name)at(0.0,0.0)`) platziert und eingefügt über `\begin{scope}[]`. Außerdem werden hier Rotation, Tikzgröße und die Platzierung eingefügt. Der Befehl `every node/.style={opacity=0}` entfernt die Beschriftungen in den Tikz für eine bessere Übersicht.

```
\begin{figure}
   \begin{center}
      \begin{tikzpicture}
         \ArduinoNanoShieldTikz
         \coordinate (BoardOrigin) at (6.8,28.1);
         \coordinate (BoardOriginNano) at (2.4,29.4);

         \begin{scope}[scale=0.65, rotate=-90,
         shift={(BoardOrigin)},
         every node/.style={opacity=0}]
            \CameraOV
         \end{scope}

         \begin{scope}[scale=0.48, rotate=-90,
         shift={(BoardOriginNano)},
         every node/.style={opacity=0}]
            \ArduinoNanoBLESenseRev
         \end{scope}

         \end{tikzpicture}
      \caption[Hardwareanbindung an das TinyML Shield]
      {Integration
```

```
      des Arduino Nano 33 BLE Sense und der Kamera OV7675
      auf dem TinyML Shield}
      \label{TinyMLShield:Configuration}
   \end{center}
\end{figure}
```

Teil IX.

Tabellen

10. Tabellen in Text einbinden

Der Syntax von Tabellen kann dem Kapitel BOM und SBOM entnommen werden. Hier ist ein Beispiel:

```
\begin{tabular}{|l|c|l|r|}
   \hline
   Description & Qty & Link & Price \\
   \hline

   Arduino Kit & 1 & reichelt.de & 50.80 \\
   RGB LED & 2 & reichelt.de & 1.98 \\

   \hline
   Total & & & 68.67 \\
   \hline
\end{tabular}
```

Description	Qty	Link	Price
Arduino Kit	1	reichelt.de	50.80
RGB LED	2	reichelt.de	1.98
Total			68.67

Teil X.

BOM und SBOM

11. Bill of Materials

Die Bill of Materials wird Berichten und Projektarbeiten im Anhang eingefügt. Wichtig bei dieser BOM ist:

- Die Tabelle sollte nicht zu weit über den Textrand rüber ragen.
- Die implementierten Links sollten, wenn möglich zu Artikeln der Firma Reichelt führen. Falls sie dort nicht vorhanden sind, bietet Conrad eine Alternative.
- Für Filamente kann Bambu angegeben werden.

Tabelle 11.1.: Hardware Bill of Materials

Image	**Qty**	**Description**	**Supplier**	**Price (€)**
	1	Arduino Tiny Machine Learning Kit	reichelt.de	50.80
Included in TinyML Kit:				
– – – –		Arduino Nano 33 BLE Sense (microcontroller) OV7675 Camera Module (640×480 VGA) Grove Shield for Arduino Nano USB micro Cable		
	2	140C05 RGB LED Module (3-color, 5 V)	reichelt.de	1.98
	1	Mini Arcade Button with Microswitch (33 mm, black)	reichelt.de	2.00

Continued on next page

Table 11.1 continued from previous page

Image	Qty	Description	Supplier	Price (€)
	1	Toggle Switch, single pole (black)	reichelt.de	0.78
	1	MAX Alkaline Batterie 9 V	reichelt.de	2.40
	2	Kunststoff-Halbschale 123 × 70 × 26 mm (ABS, grau)	reichelt.de	1.00
	1	Batteriehalter für 1× 9 V-Block	reichelt.de	6.99
	1	Grove Universal-Kabel, 4-Pin, 5 cm (5er-Pack)	reichelt.de	1.70
			Total	**≈ 68.67**

12. Software Bill of Materials

In der Software Bill of Materials wird die verwendete Software aufgelistet. Diese wird dem Bericht als Anhang ergänzt. Es gelten die gleichen Regeln, wie bei der BOM. Die Links sollten zu den offiziellen Websites der Software führen.

Tabelle 12.1.: Software Bill of Materials

Software	Description	Installation Source	Version	License	Cost
Python	High-level programming language used for model training and preprocessing	python.org	3.11	PSF License	Free
NumPy	Numerical computing and array processing for image data and tensors	numpy.org	≥1.24	BSD License	Free
OpenCV (Python)	Image loading, conversion, resizing, and preprocessing	opencv.org	≥4.8	Apache 2.0	Free
TensorFlow	Deep learning framework for model training and optimization	tensorflow.org	≥2.12, <2.16	Apache 2.0	Free
TensorFlow Lite	Model conversion and optimization for embedded deployment	tensorflow.org/lite	Included	Apache 2.0	Free
Scikit-learn	Feature analysis, PCA visualization, and data splitting	scikit-learn.org	≥1.3	BSD License	Free
Matplotlib	Visualization of datasets, embeddings, and evaluation results	matplotlib.org	≥3.7	Matplotlib License	Free

Continued on next page

Table 12.1 continued from previous page

Software	Description	Installation Source	Version	License	Cost
Pillow	Image file handling and format conversion	python-pillow.org	≥10.0	HPND	Free
PySerial	Serial communication between Arduino and host system	pyserial.readthedocs.io	≥3.5	BSD License	Free
Arduino IDE	Development environment for embedded firmware	arduino.cc	2.x	GPL/ LGPL	Free
pytest	Unit testing framework for Python preprocessing and training code	pytest.org	≥7.4	MIT License	Free
Visual Studio Code	Code editor for Python and embedded development	code.visualstudio.com	Latest	MIT License	Free
Windows /macOS/ Linux	Host operating system for development and training	Vendor websites	N/A	Proprietary /OSS	N/A

Teil XI.

Templates

13. Sensor XY/ Actor XY

Introduction

WS:cite books, applications, board

13.1. General

General description
cite books

13.2. Specific Sensor/Actor

cite board

13.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

13.4. Library

13.4.1. Description

13.4.2. Installation

- data types
- data structure
- data quantity
- data quality

13.4.3. Functions

13.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

13.5.1. Manual

13.5.2. Inside the Example

13.5.3. Code

13.5.4. Files

13.6. Calibration

cite method

13.7. Simple Code

13.8. Simple Application

13.9. Tests

13.9.1. Simple Function Test

13.9.2. Test all Functions

13.10. Further Readings

14. Sensor XY/ Aktor XY

Einleitung

WS:Bücher zitieren, Anwendungen, Board

14.1. Allgemeines

- Allgemeine Beschreibung
- Bücher zitieren - Allgemeine Verwendung des Sensors
- Ableitung der Eigenschaften des Sensors bsp. Temperatur
- Überblick über verschiedene Arten von Temperatursensoren
- Typische Einsatzgebiete von Temperatursensoren
- Quellenangaben zu konkreten Anwendungen des verwendeten Sensors
- Besondere Merkmale des Sensors
  - Einsatz nur in bestimmten Anwendungsbereichen
  - Einsatz nur in bestimmten Ländern oder Märkten
  - Einsatz nur innerhalb definierter Temperaturbereiche
- (Am besten feststellen unser Sensor ist genial für unseren Anwendungszweck)

14.2. Spezifischer Sensor/Aktor

Ist der Sensor auf einer Platine? Trenne hier die Beschreibung vom Sensor Board und dann Sensor Spezifikation

- Board zitieren
- Besondere Eigenschaften des Sensors
- Begründung der Sensorwahl
- Funktionsumfang und Leistungsfähigkeit des Sensors
- Bewertung der Sensorwahl im Hinblick auf den Anwendungsfall
- Zitierung des Datenblatts
- Möglichkeit der eindeutigen Identifizierbarkeit des Sensors
- Kabel- und Anschlussmerkmale (z. B. IP-Schutzklasse) als Indikator für die Robustheit
- Abbildung des Sensors

14.3. Spezifikation

- Datenblatt zitieren
- Informationen aus dem Datenblatt extrahieren
- Schaltplan

14.4. Bibliothek

- Arduino Bibliothek oder Python package für den Sensor vorhanden?

14.4.1. Beschreibung

14.4.2. Installation

- Datentypen
- Datenstruktur
- Datenmenge
- Datenqualität

14.4.3. Funktionen

14.5. Beispiel

In diesem Abschnitt wird ein einfaches Beispiel detailliert beschrieben, das demonstriert, wie die Bibliothek den Sensor/Aktor unterstützt.

14.5.1. Anleitung

14.5.2. Innerhalb des Beispiels

14.5.3. Code

14.5.4. Dateien

14.6. Kalibrierung

Methode zitieren

14.7. Einfacher Code

14.8. Einfache Anwendung

14.9. Tests

14.9.1. Einfacher Funktionstest

14.9.2. Test aller Funktionen

14.10. Weiterführende Literatur

Literatur

[Ard23] Arduino. *Get Started With Machine Learning on Arduino | Arduino Documentation*. 2023. URL: https://docs.arduino.cc/tutorials/nano-33-ble-sense/get-started-with-machine-learning.

[Ard25] Arduino. *Arduino Nano 33 BLE Sense Lite Documentation*. 2025. URL: https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2/.

[BS22] M. Banzi und M. Shiloh. *Getting started with Arduino*. Maker Media, Inc., 2022. ISBN: 9780596555108.

Index

Abb, ix, 21
Abbidung
    siehe Abb, ix, 21